

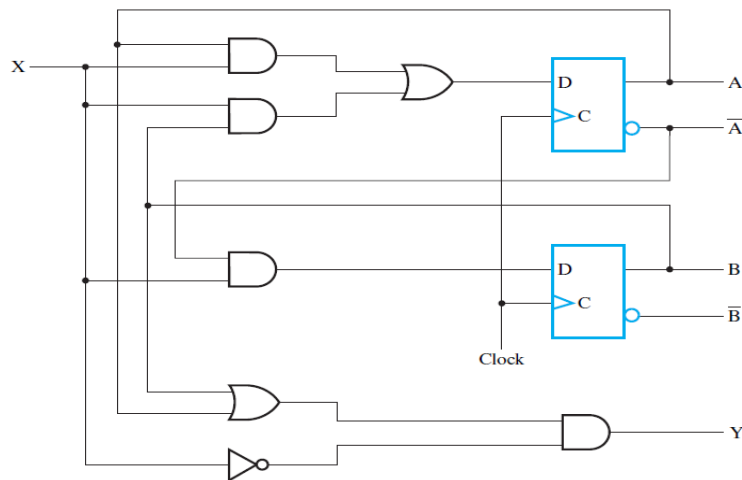


Finite State Machines (FSMs): Formal Modeling of Sequential Systems



1) Formal Modeling of Sequential Systems

Abstracted
as FSM

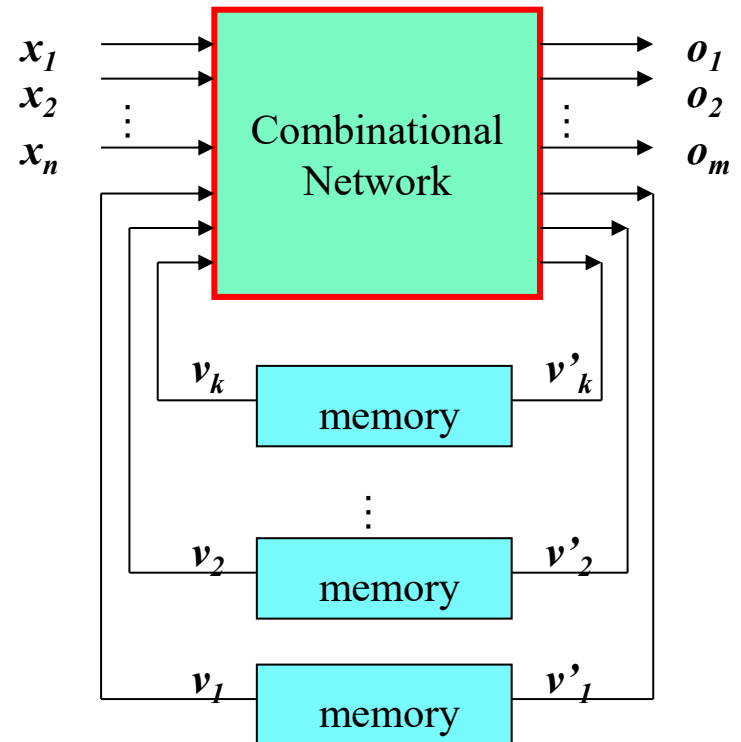
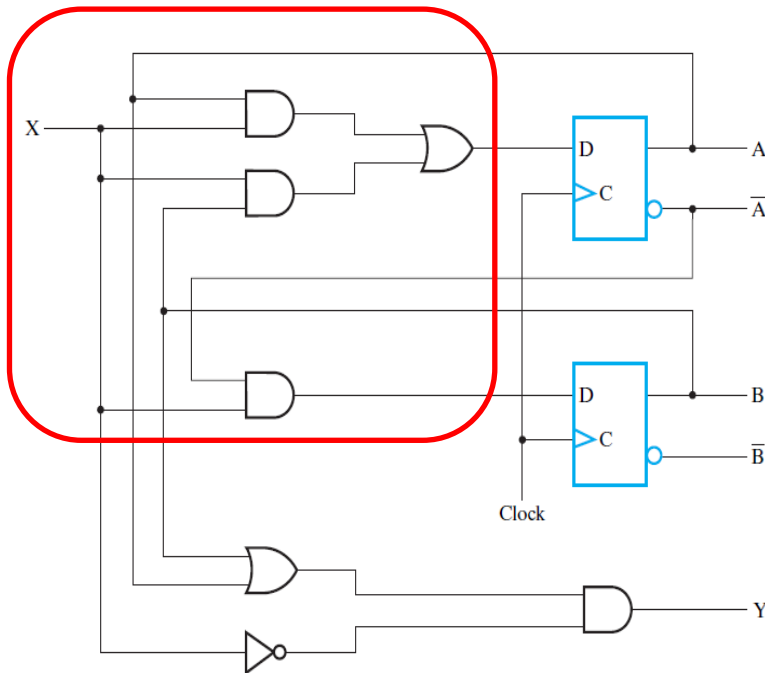


- $M = (I, S, \delta, \lambda, O, S_0)$ where
 - I : set of inputs
 - S : set of states
 - O : set of outputs
 - δ : transition relation ($\delta \in S \times I \times S$)
 - λ : output relation
 - S_0 : set of initial states
 - Deterministic machines: δ and λ are functions, $S_0 = \{s_0\}$.

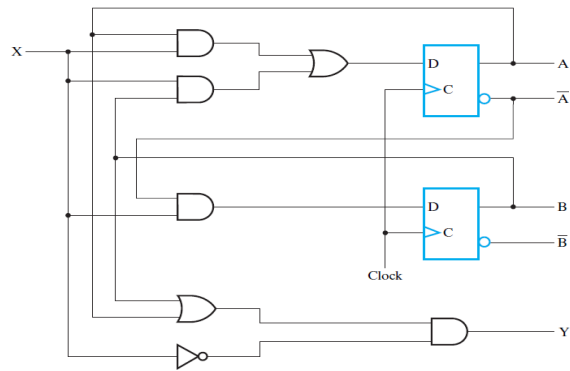
- A state characterizes the behavior of a sequential system, given a fixed set of values of the memory elements

Next State Function: Driving Engine

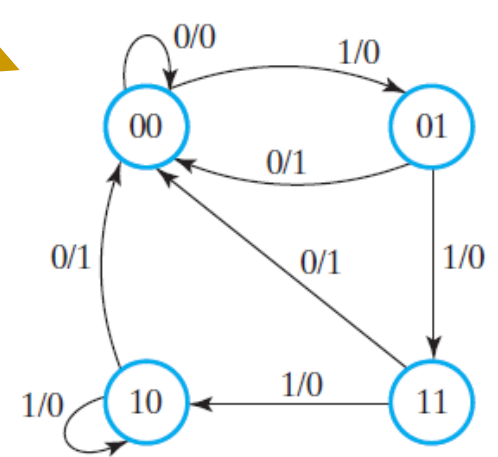
- The current state $V = (v_1, v_2, \dots, v_n)$
- The next state $V' = (v'_1, v'_2, \dots, v'_n)$
- Next State Function: $V' = (v'_1, v'_2, \dots, v'_n) = R(v_1, v_2, \dots, v_n)$



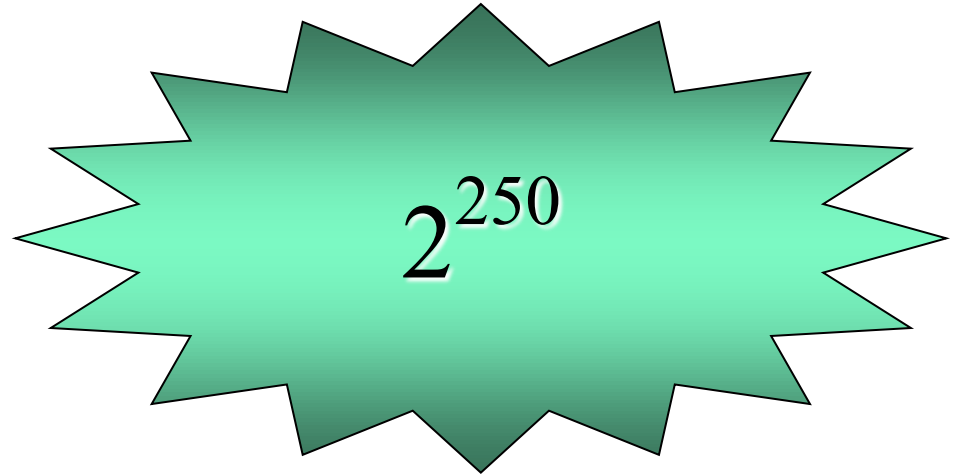
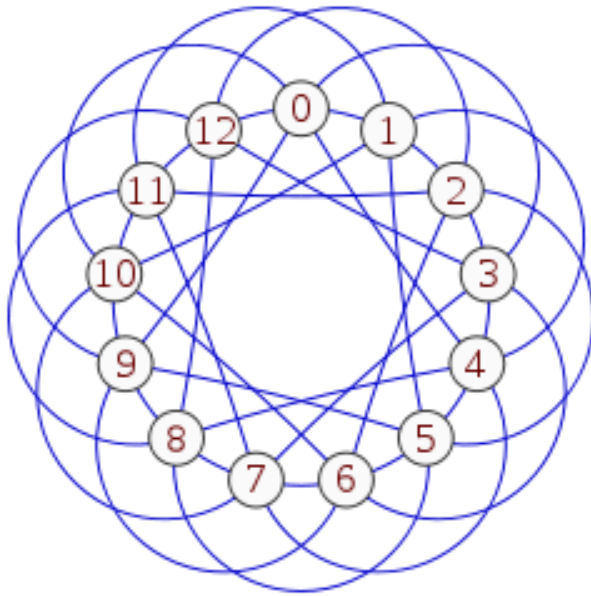
State Transition Graph (STG)



Present State		Input	Next State		Output
A	B	X	A	B	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0



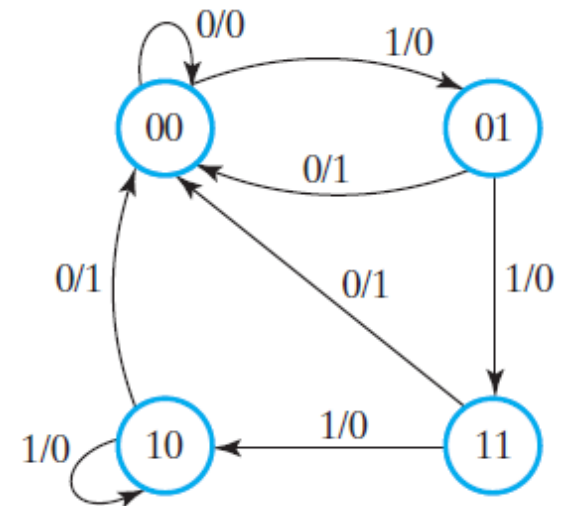
State Explosion Problem



- Problem: "State explosion" - e.g., 32-bit $\rightarrow$ register 2^{32} states
- A small design that uses 250 bits of data has at least 2^{250} states
 $\Rightarrow$ More states than particles in the Universe!
- The model is too large to store in available memory!

Foundation for Automatic Formal Sequential Verification

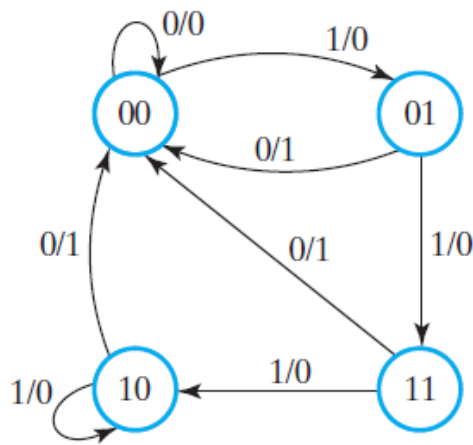
Present State		Input	Next State		Output
A	B	X	A	B	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0



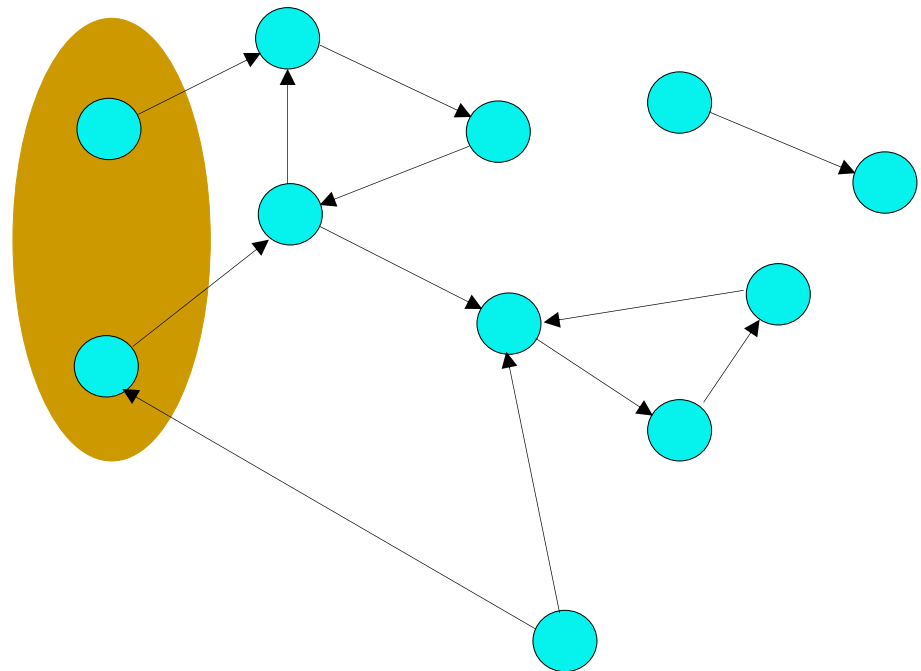
- State Reachability Analysis (State Enumeration, State Traversal)

State Reachability Analysis (State Enumeration, State Traversal)

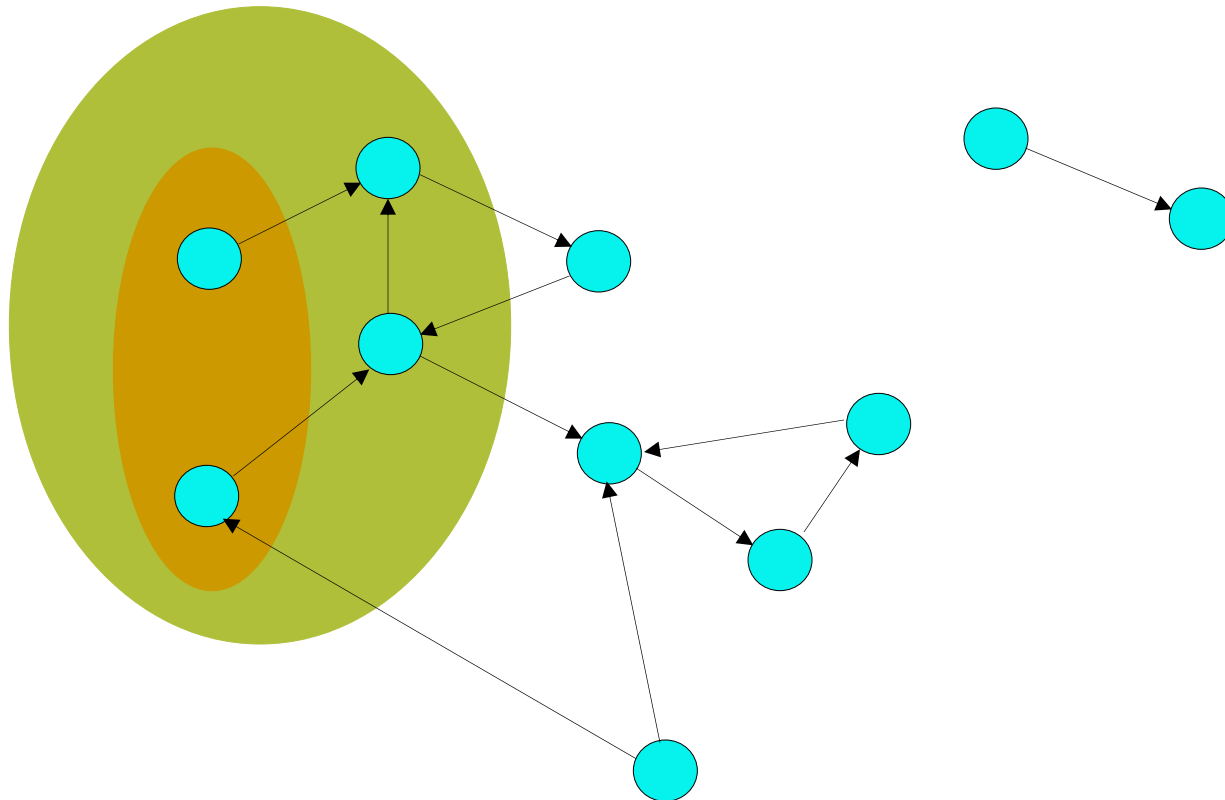
Present State		Input	Next State		Output
A	B	X	A	B	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0



S_0 = the set of initial states

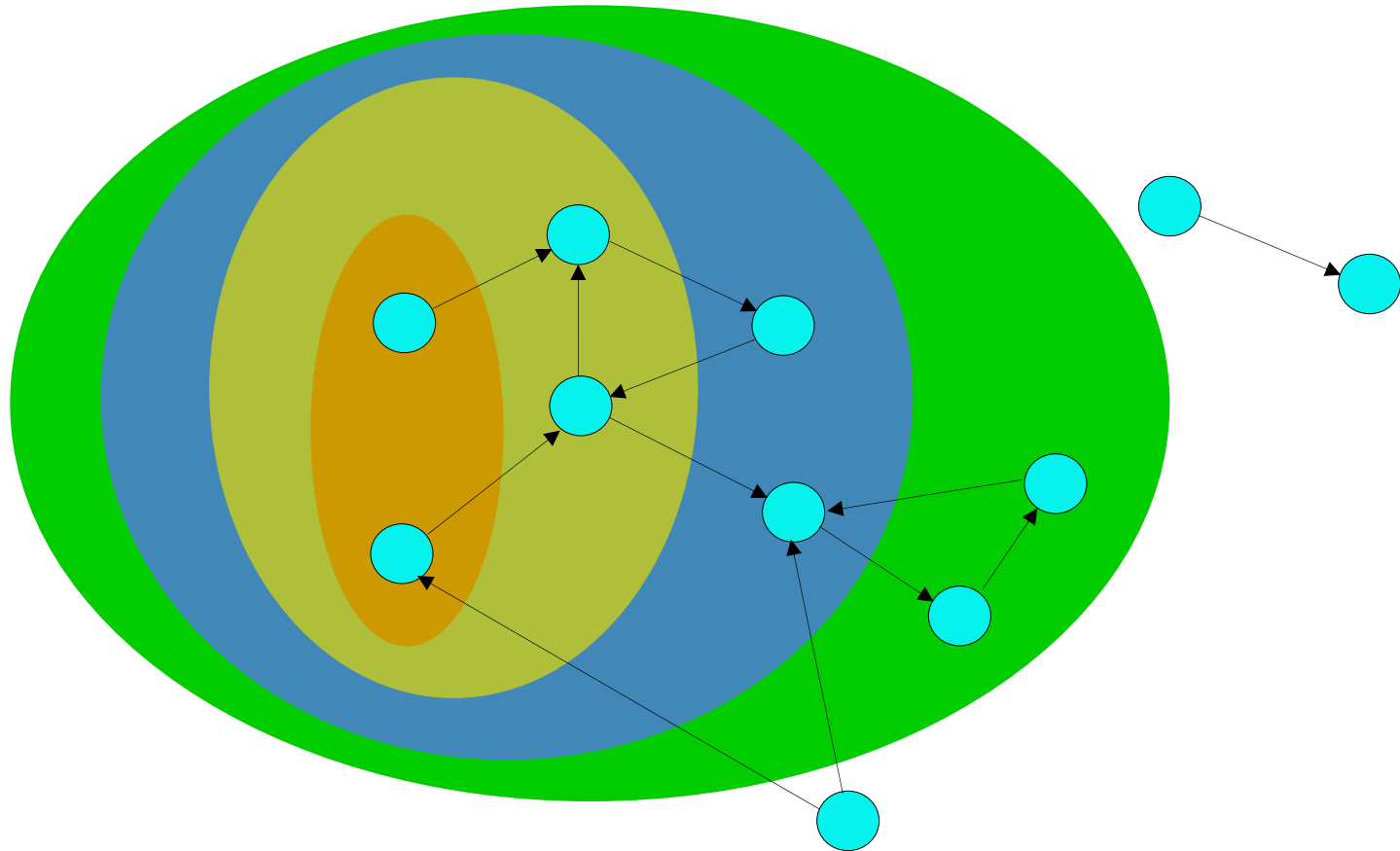


Set of Reachable States



S_1 is the set of states reachable from the initial states in less than or equal to one step.

Fixed-point Set of Reachable States

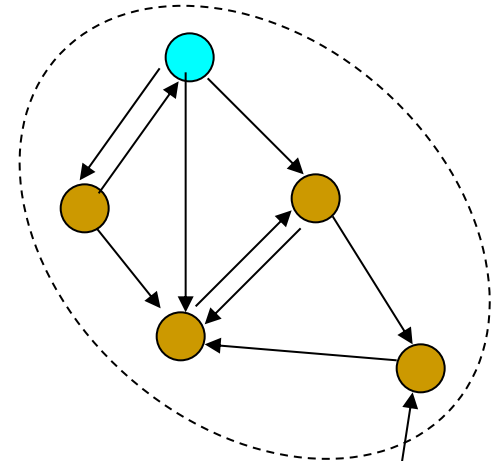


The fixed-point iteration terminates when it reaches $S_6 = S_5$, and the resulting set is the set of reachable states.

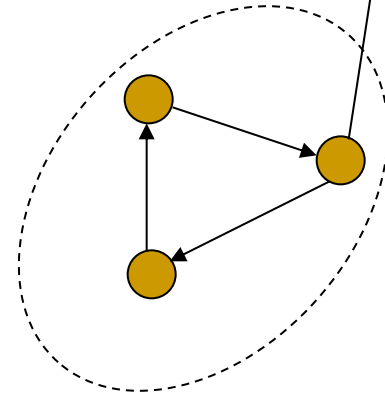
State Reachability Analysis

- Initial states:
 - A state, in which the system begins its functioning.
- Reachable states:
 - A state that can be reached from the initial one through a finite sequence of transitions under allowed inputs.
- Unreachable states:
 - A state that cannot be reached from the initial under any sequence of inputs.

Reachable states



Unreachable states



State Reachability Analysis Ideas

- Reachability(Transition Relation **R**, Initial State I)
{
 ReachedStates = I;
 iterate the following computation:
 {
 ReachedStatesNew = Image(TR, ReachedStates);

 if (ReachedStatesNew is contained in ReachedStates) stop;

 ReachedStates = ReachedStates + ReachedStatesNew;
 }
}

A Detailed Reachability Analysis

Reachability (R, I) {

1. **ARS = NNS = I;**
2. **Iterate the following computation: {**
3. **NS = R(all inputs, NNS);**
4. **NNS = NS - ARS;**
5. **if NNS = $\emptyset$ then print ARS and stop;**
6. **ARS = ARS $\cup$ NNS; }**
- }**

R: Next state function: from current states to next states

I: The set of the initial states.

NS: The set of the reached next states.

NNS: The set of the new next states (called Frontier Set).

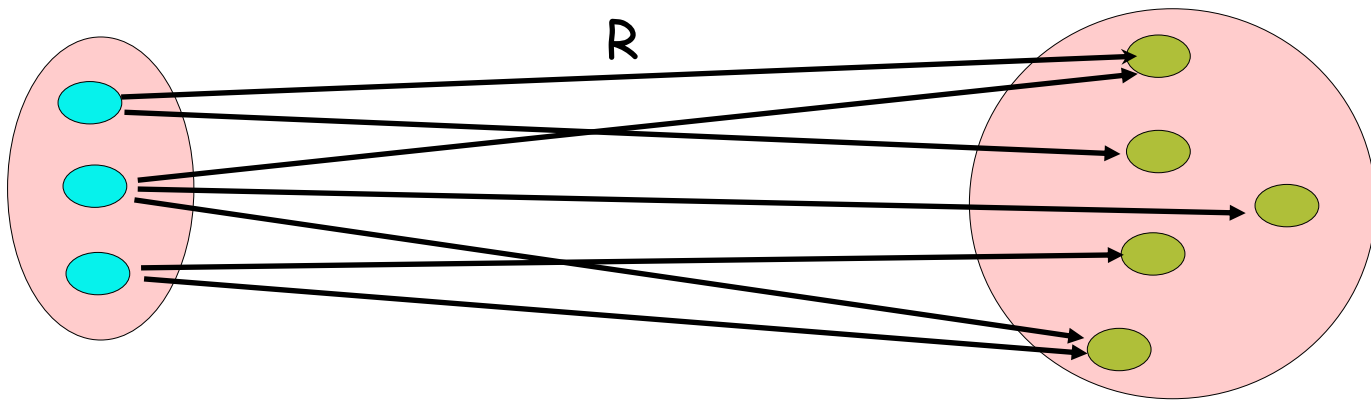
ARS: The set of all the reached states.

$NS = R(\text{all inputs}, NNS)$

Reachability (R, I) {

1. $ARS = NNS = I$;
2. Iterate the following computation: {
 3. $NS = R(\text{all inputs}, NNS)$;
 4. $NNS = NS - ARS$;
 5. if $NNS = \emptyset$ then print ARS and stop;
 6. $ARS = ARS \cup NNS$;}

}

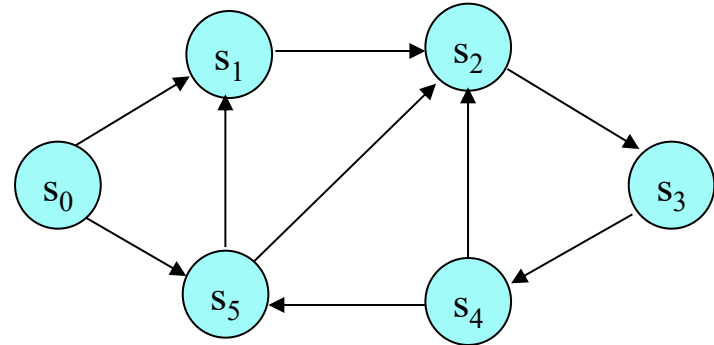


Explicit State Enumeration

Reachability (R, I) {

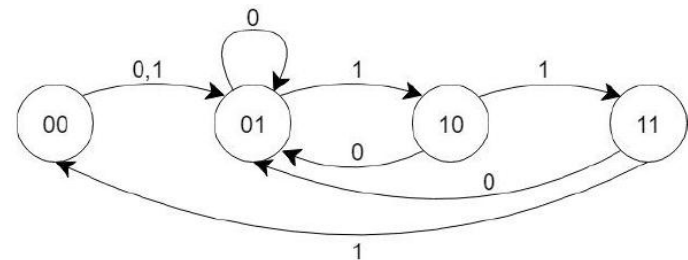
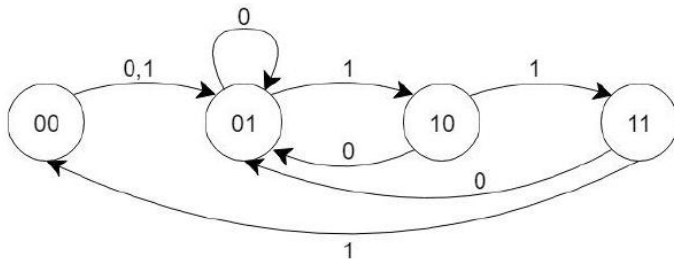
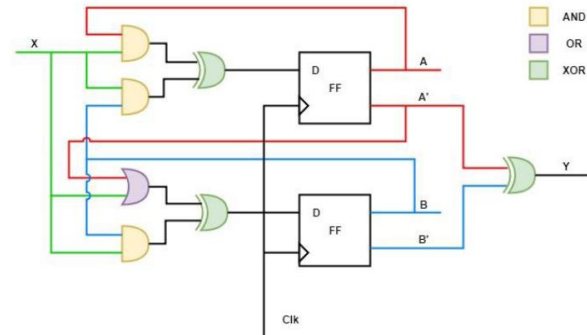
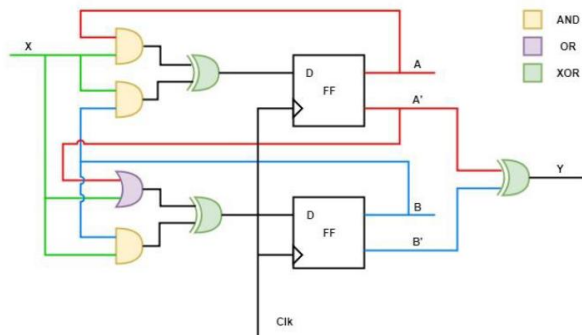
1. $ARS = NNS = I$;
2. Iterate the following computation: {
3. $NS = R(\text{all inputs}, NNS)$;
4. $NNS = NS - ARS$;
5. if $NNS = \emptyset$ then print ARS and stop;
6. $ARS = ARS \cup NNS$; }

}



	NS	NNS	ARS
Initialization		{S ₀ }	{S ₀ }
Iteration 1	{S ₁ , S ₅ }	{S ₁ , S ₅ }	{S ₀ , S ₁ , S ₅ }
Iteration 2	{S ₁ , S ₂ }	{S ₂ }	{S ₀ , S ₁ , S ₂ , S ₅ }
Iteration 3	{S ₃ }	{S ₃ }	{S ₀ , S ₁ , S ₂ , S ₃ , S ₅ }
Iteration 4	{S ₄ }	{S ₄ }	{S ₀ , S ₁ , S ₂ , S ₃ , S ₄ , S ₅ }
Iteration 5	{S ₂ , S ₅ }	{ }	

2) Sequential Equivalence Checking

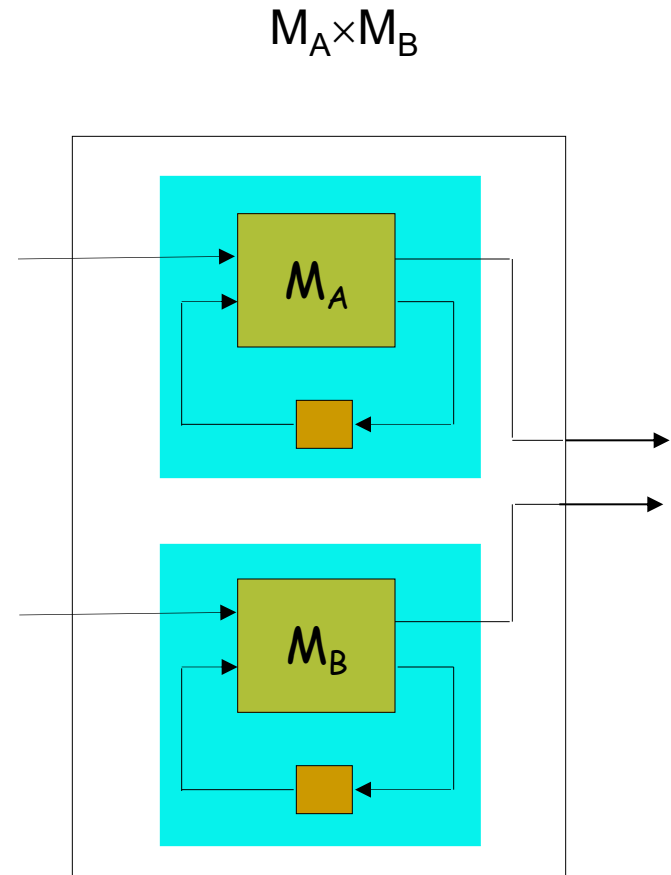


Product Machine

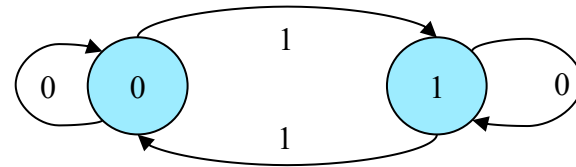
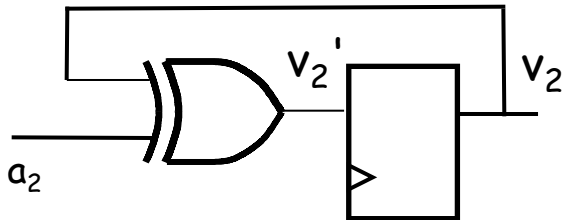
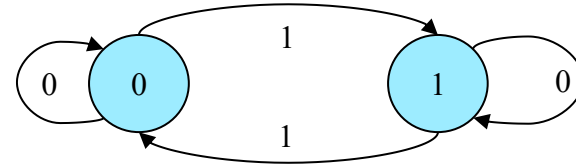
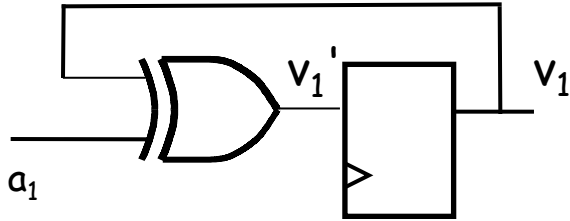
- Two sets $A = \{a, b, c\}$ and $B = \{1, 2\}$.
- $A \times B = \{ \langle a, \beta \rangle \mid a \in A \text{ and } \beta \in B \}$
- $A \times B = \{ \langle a, 1 \rangle, \langle a, 2 \rangle, \langle b, 1 \rangle, \langle b, 2 \rangle, \langle c, 1 \rangle, \langle c, 2 \rangle \}$
- $\text{FSM } A = (S_A, I_A, O_A, R_A, I_A)$
- $\text{FSM } B = (S_B, I_B, O_B, R_B, I_B)$
- Assumptions:
 - Synchronized: orchestrated by the same clock
 - $A \cap B = \emptyset$: they use different variables

Product Machine

- $\text{FSM } A = (S_A, I_A, O_A, R_A, I_A)$
- $\text{FSM } B = (S_B, I_B, O_B, R_B, I_B)$
- $\text{FSM } C = A \times B = (S_C, I_C, O_C, R_C, I_C)$
 - $S_C = S_A \times S_B$
 - $I_C = I_A \times I_B$
 - $O_C = O_A \times O_B$
 - $I_C = (I_A, I_B)$
 - $R_C((s_A, s_B), (i_A, i_B)) = ((s'_A, s'_B), (o_A, o_B))$,
 - where
 - $(s'_A, o_A) = R_A(s_A, i_A)$, and $(s'_B, o_B) = R_B(s_B, i_B)$,
 - for all $s_A \in S_A, s_B \in S_B, i_A \in I_A$, and $i_B \in I_B$.



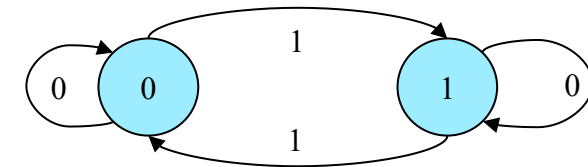
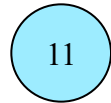
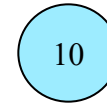
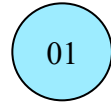
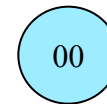
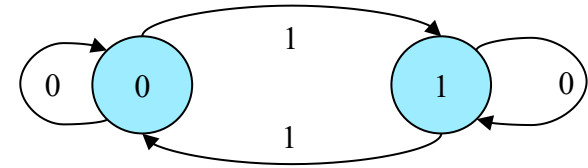
M_1 and M_2



Product Machine

$$v = (v_1, v_2); v' = (v_1', v_2')$$

a_1	a_2	v_1	v_2	v_1'	v_2'
0	0	0	0	0	0

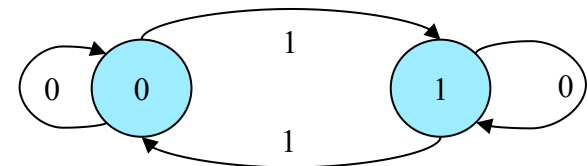
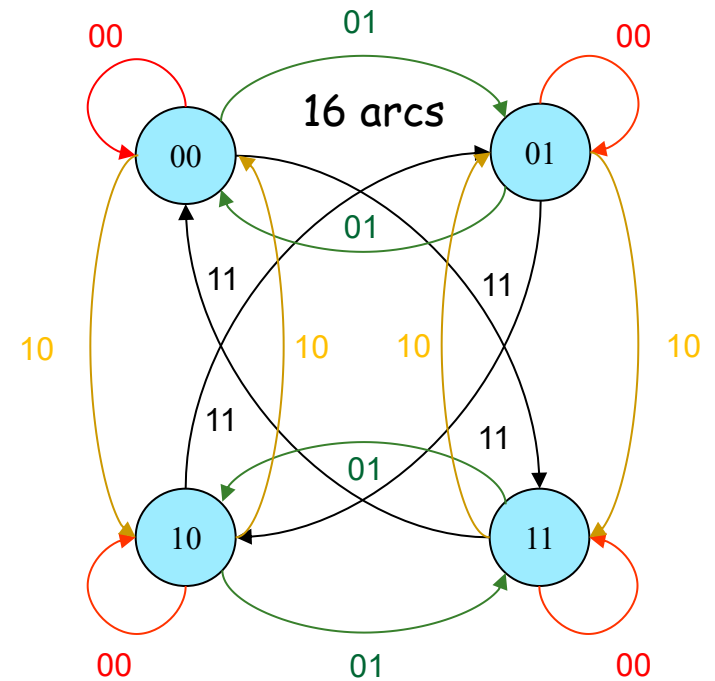
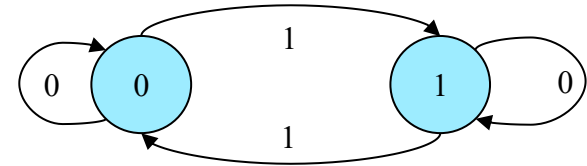


Product Machine

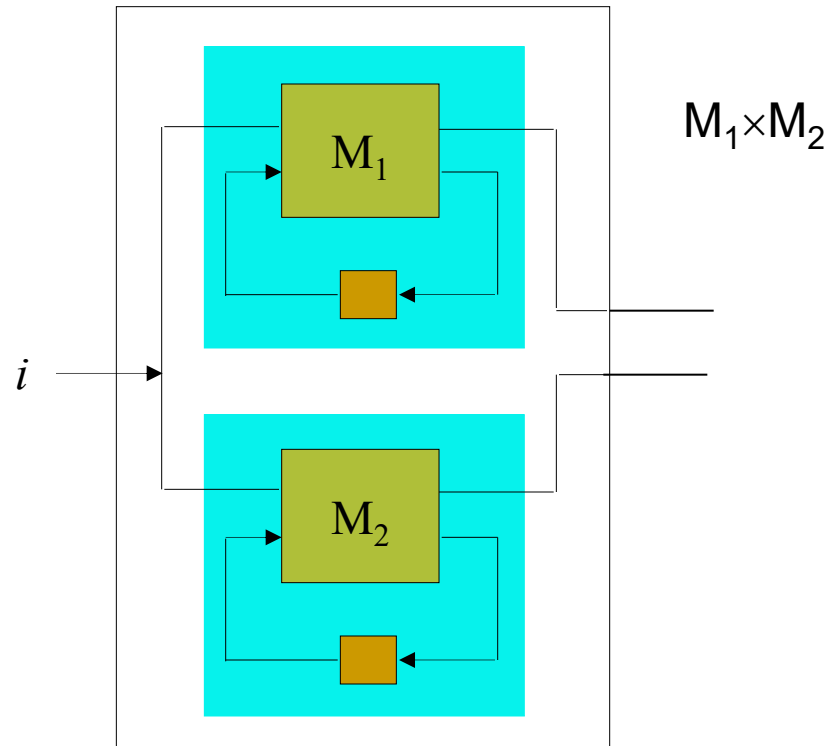
$$v = (v_1, v_2); v' = (v_1', v_2')$$

a_1	a_2	v_1	v_2	v_1'	v_2'
0	0	0	0	0	0
0	0	0	1	0	1
0	0	1	0	1	0
0	0	1	1	1	1
0	1	0	0	0	1
0	1	0	1	0	0
0	1	1	0	1	1
0	1	1	1	1	0

16 lines

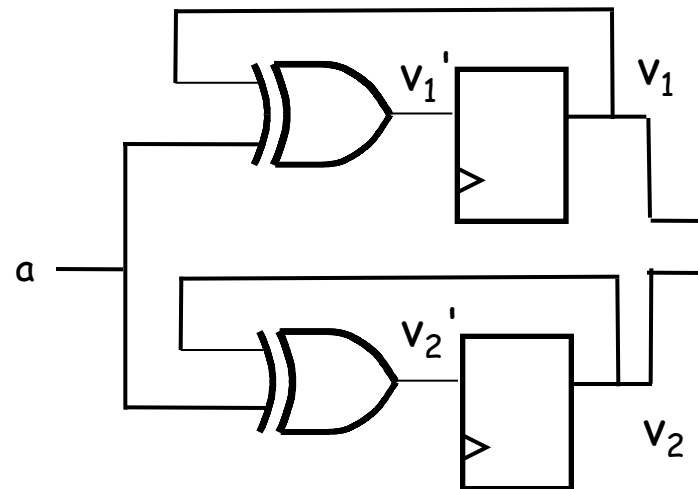
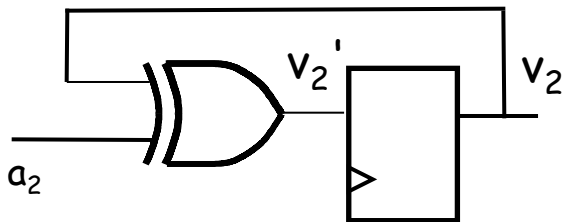
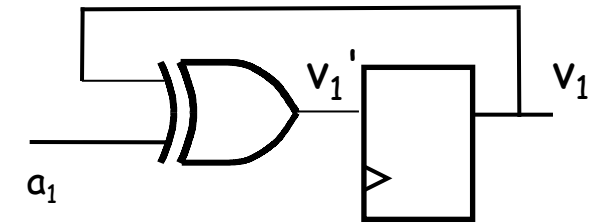


Shared-Input Product Machine



- $v = (v_1, v_2); v' = (v'_1, v'_2), i = i_1 = i_2$
- $R(v, x, v') = R_1(v_1, x, v'_1) \wedge R_2(v_2, x, v'_2)$

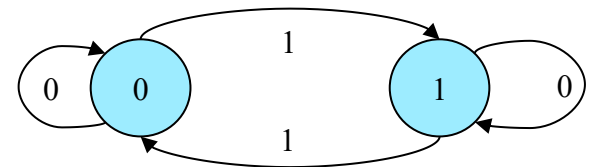
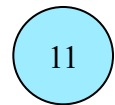
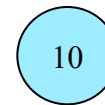
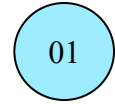
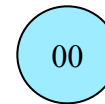
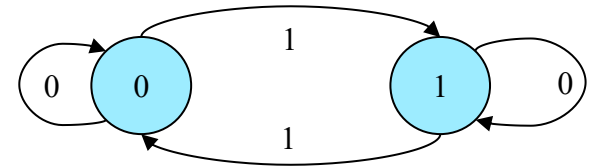
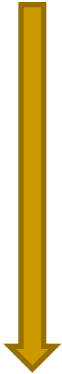
Shared-Input Product Machine



Shared-Input Product Machine

$$v = (v_1, v_2); v' = (v_1', v_2')$$

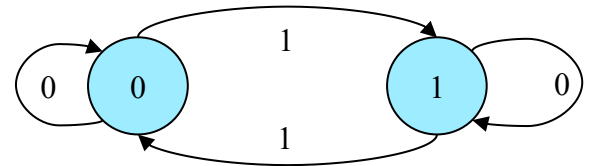
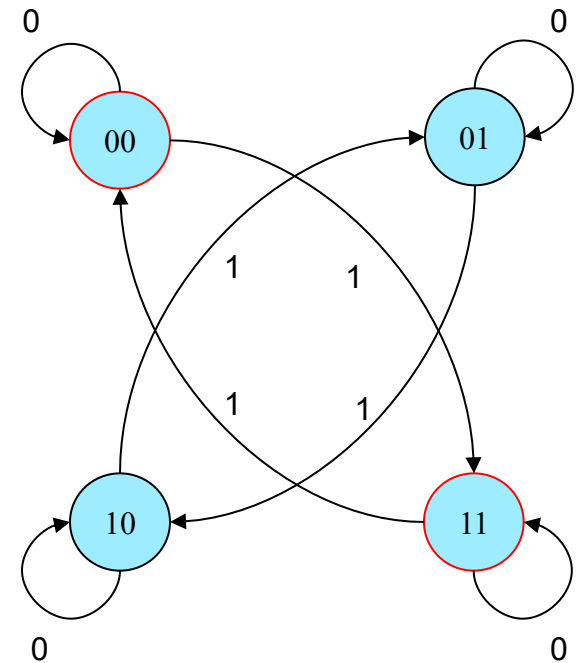
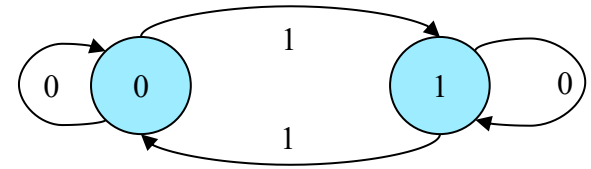
a	v₁	v₂	v₁'	v₂'
0	0	0	0	0



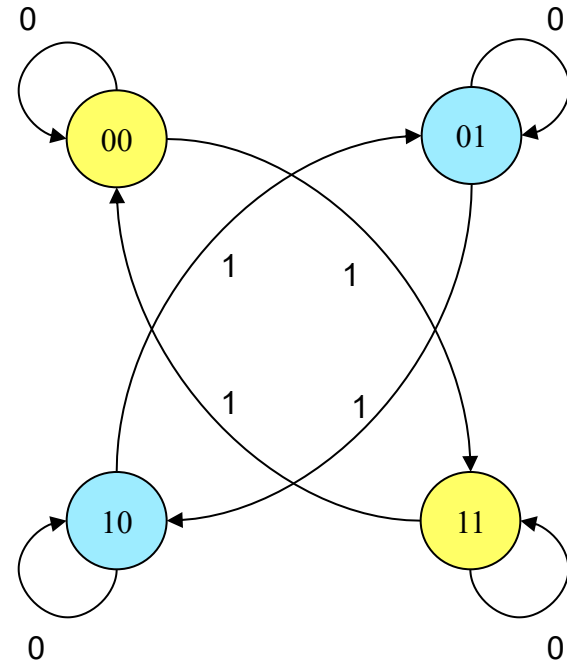
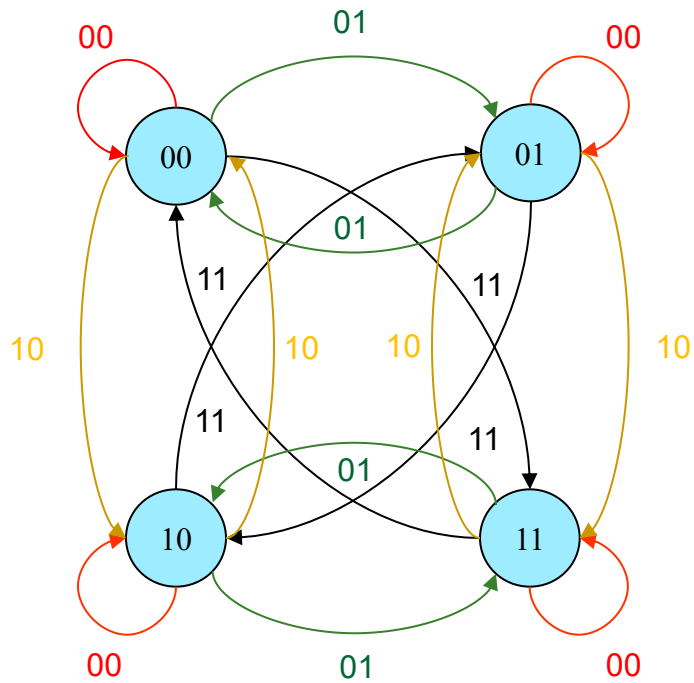
Equivalence Checking

$$v = (v_1, v_2); v' = (v_1', v_2')$$

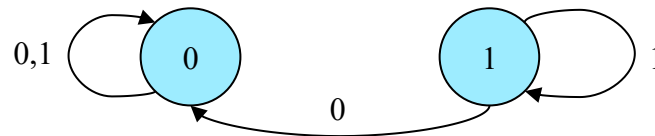
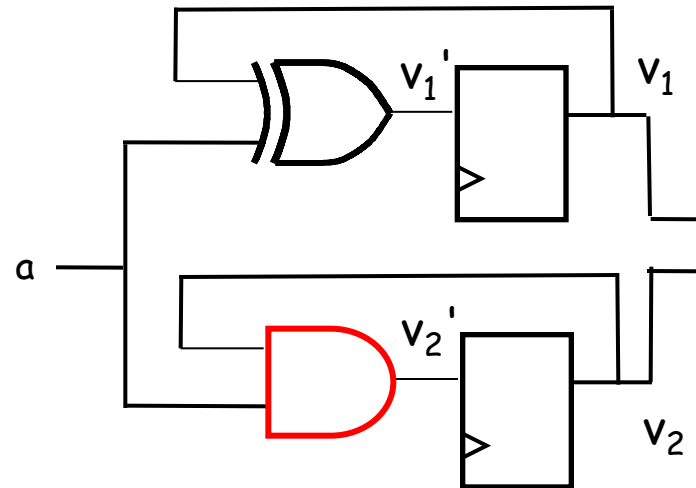
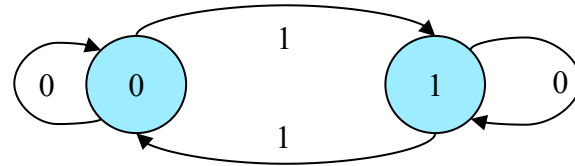
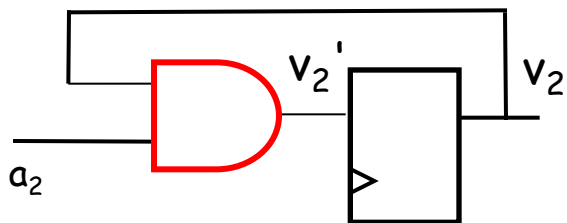
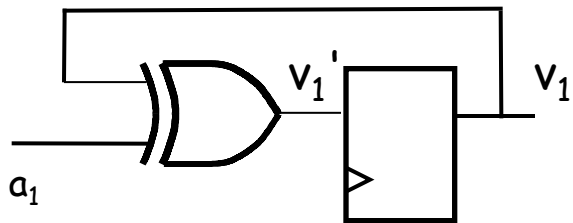
a	v₁	v₂	v₁'	v₂'
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	1	1
1	0	0	1	1
1	0	1	1	0
1	1	0	0	1
1	1	1	0	0



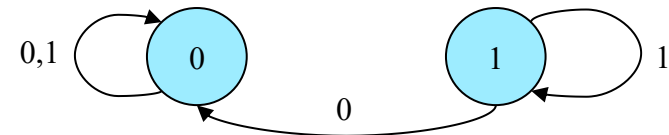
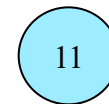
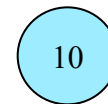
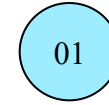
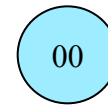
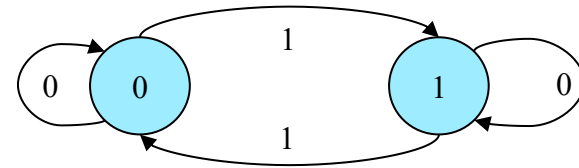
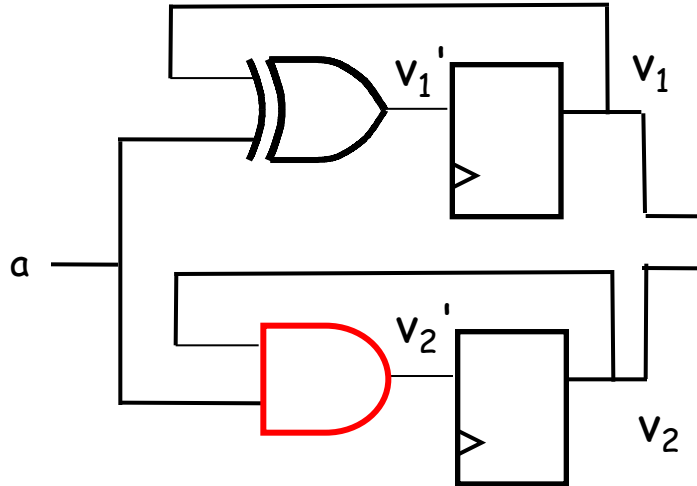
Difference



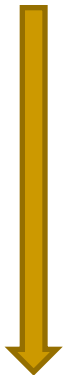
Shared-Input Product Machine



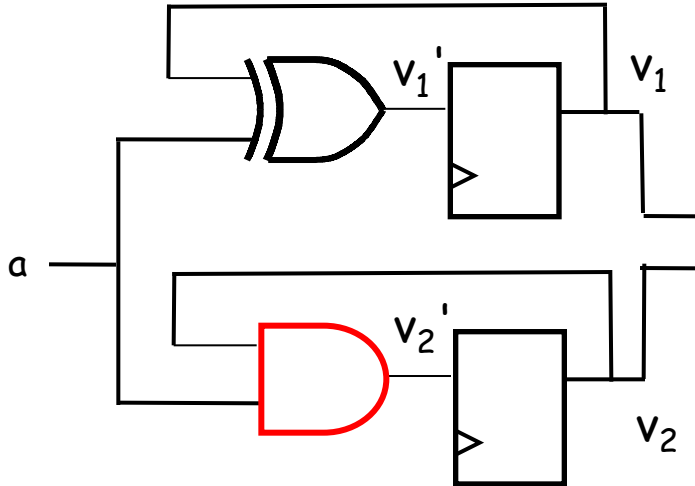
Shared-Input Product Machine



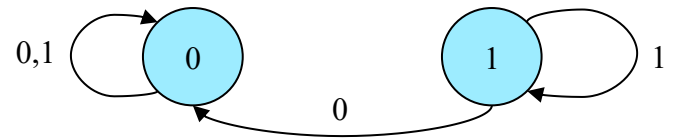
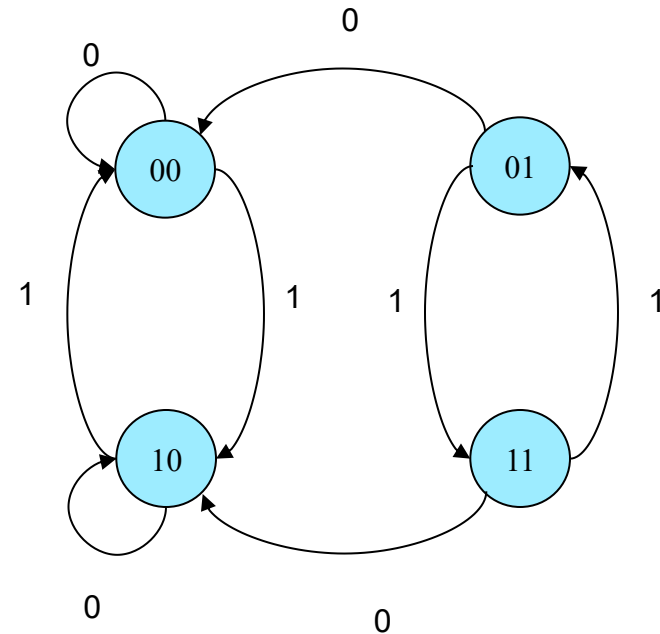
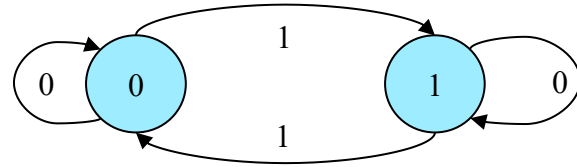
a	v₁	v₂	v₁'	v₂'
0	0	0	0	0



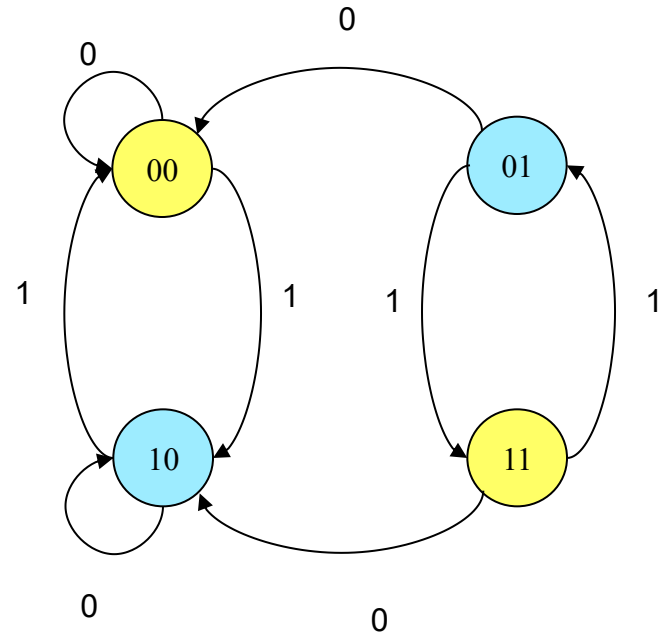
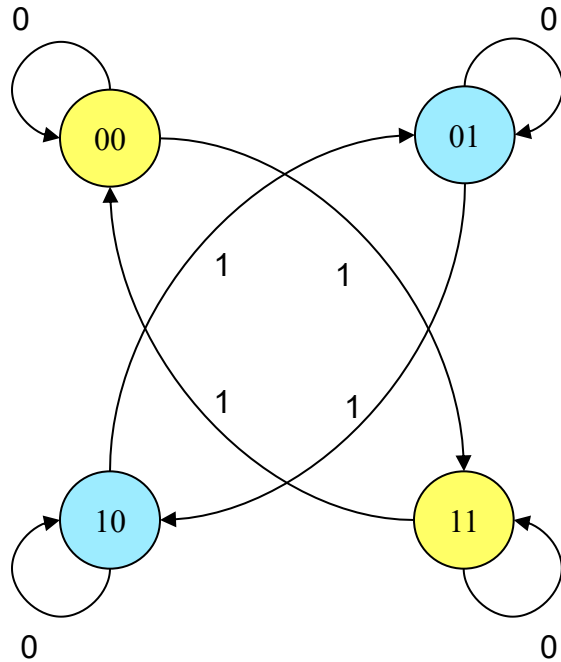
Equivalence Checking



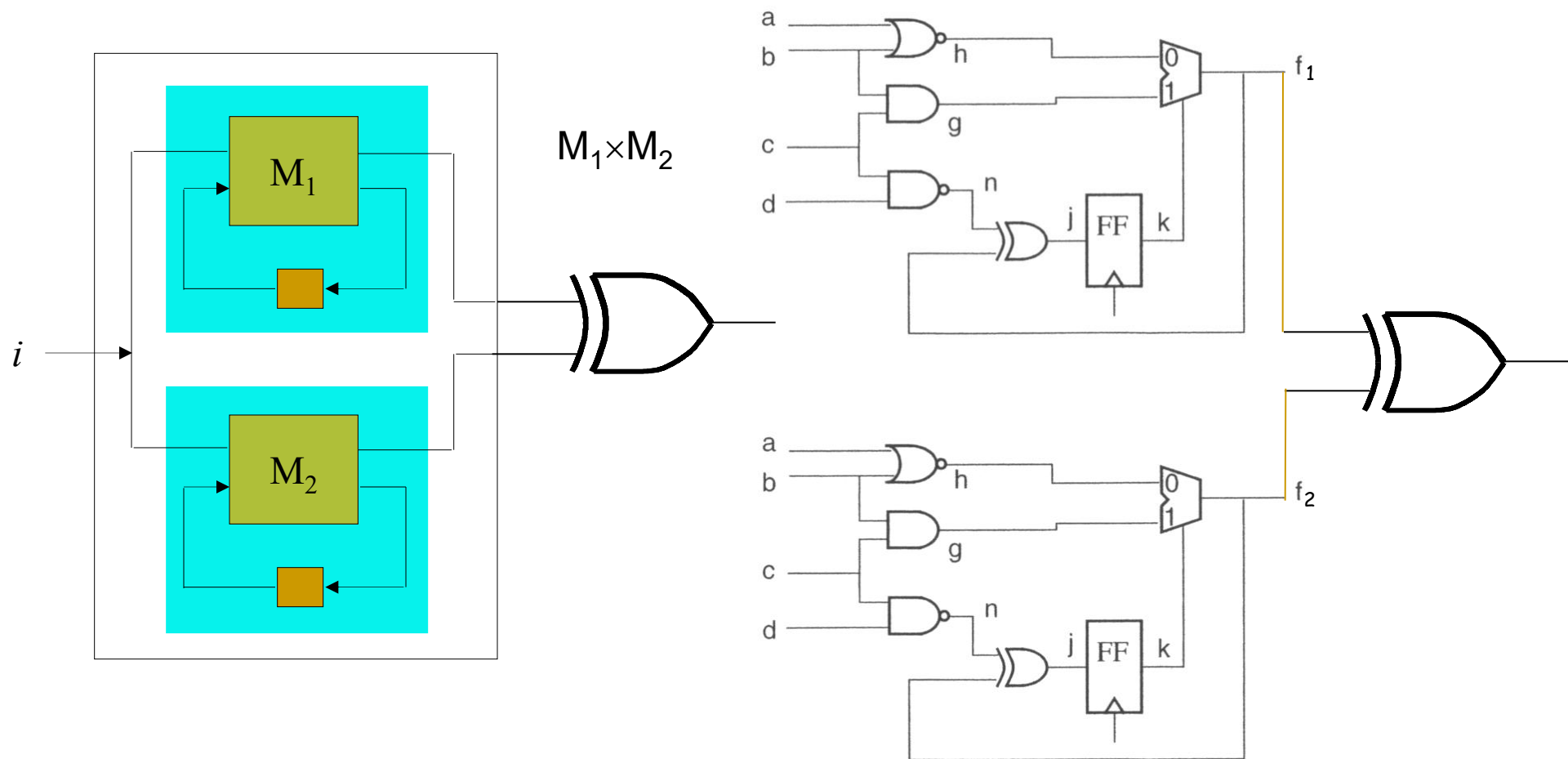
a	v₁	v₂	v₁'	v₂'
0	0	0	0	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	0	0
1	1	1	0	1



Equivalence Checking



Shared-Input Product Machine



References

- K. L. McMillan. *Symbolic Model Checking*. Kluwer Academic Publishers, 1993.
- J. R. Burch, E. M. Clarke, D. Long, K. L. McMillan, D. L. Dill. Symbolic model checking for sequential circuit verification. *IEEE Transactions on CAD*, 13(4), 1994, pp. 401-424.
- J. R. Burch, E. M. Clarke, K. L. McMillan, D. L. Dill, and L. J. Hwang. Symbolic model checking: 10^{20} states and beyond. *Information and Computation*, 98(2), June 1992.
- E. M. Clark, O. Grumberg and D. A. Peled, "Model Checking", MIT Press, 1999.
- M. Huth, M. Ryan, "Logic in Computer Science: Modeling and Reasoning about Systems", Cambridge University Press, 2000.
- William K. Lam, "Hardware Design Verification: Simulation and Formal Method-Based Approaches", Prentice Hall, 2005.
- Edward A. Lee and Sanjit A. Seshia, *Introduction to Embedded Systems, A Cyber-Physical Systems Approach*, <http://LeeSeshia.org>, ISBN 978-0-557-70857-4, 2011.